МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

Дисциплина «Структуры и алгоритмы обработки данных»

Отчет по лабораторной работе №3

Выполнила:

Студент группы БВТ2504
Мясникова Е.А.

Проверила: Шедова А.Р

Москва, 2026 г.

Содержание

Цель работы:.....2
Ход выполненной работы:.....2
1. Подготовка.....2
2. Задание 1.....2
3. Задание 2.....6
4. Задание 3.....7
5. Задание 4.....9
6. Задание 5.....10
7. Задание 6.....14
Заключение.....16

Цель работы: Изучить и реализовать сортировки пузырьком, вставками, выбором и QuickSort; проверить правильность и стабильность их работы, исследовать количество операций и зависимость времени выполнения от размера и первоначального порядка элементов массива.

Ход выполненной работы:

1. **Подготовка**

Работа выполнялась в PyCharm с использованием Python и виртуального окружения .venv. В папке student_work/lab03 был создан файл lab03.py на основе заготовки simple03.py. Для варианта 16 использовалось значение seed = 46, чтобы получать воспроизводимые случайные данные.

2. **Задание 1**

Были реализованы сортировки пузырьком, вставками и выбором. Каждая функция возвращала отсортированную копию массива, количество сравнений и количество обменов или перемещений. В сортировке пузырьком был предусмотрен досрочный выход, если за проход не произошло ни одного обмена.

```python
import argparse
import random
from collections import Counter


def bubble_sort(a: list, key=lambda x: x) -> tuple[list, int, int]:  4 usages  new *
    """Пузырёк: считаются сравнения и обмены соседних элементов."""
    b = list(a)
    comparisons = swaps = 0

    for end in range(len(b) - 1, 0, -1):
        changed = False

        for j in range(end):
            comparisons += 1
            if key(b[j]) > key(b[j + 1]):
                b[j], b[j + 1] = b[j + 1], b[j]
                swaps += 1
                changed = True

        if not changed:
            break

    return b, comparisons, swaps
```

Рисунок 1 - Реализация сортировки пузырьком со счётчиками операций

```python
def insertion_sort(a: list, key=lambda x: x) -> tuple[list, int, int]:  4 usages  new *
    """Вставки: считаются сравнения и перемещения элементов. Перемещение - сдвиг элемента или запись вставляемого
    элемента на новое место. Копирование входа не учитывается.
    """
    b = list(a)
    comparisons = moves = 0

    for i in range(1, len(b)):
        value = b[i]
        value_key = key(value)
        j = i - 1

        while j >= 0:
            comparisons += 1

            if key(b[j]) <= value_key:
                break

            b[j + 1] = b[j]
            moves += 1
            j -= 1

        if j + 1 != i:
            b[j + 1] = value
            moves += 1

    return b, comparisons, moves
```

Рисунок 2 - Реализация сортировки вставками со счётчиками операций

```python
def selection_sort(a: list, key=lambda x: x) -> tuple[list, int, int]:  3 usages  new *
    """Выбор: считаются сравнения и фактически выполненные обмены."""
    b = list(a)
    comparisons = swaps = 0

    for i in range(len(b) - 1):
        minimum = i

        for j in range(i + 1, len(b)):
            comparisons += 1
            if key(b[j]) < key(b[minimum]):
                minimum = j

        if minimum != i:
            b[i], b[minimum] = b[minimum], b[i]
            swaps += 1

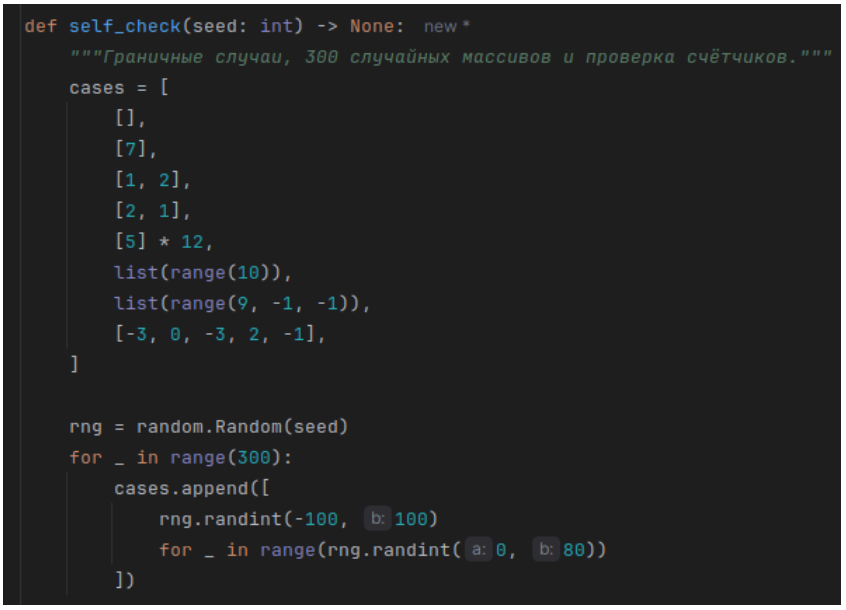
    return b, comparisons, swaps
```

Рисунок 3 - Реализация сортировки выбором со счётчиками операций

```python
ALGORITHMS = {
    "bubble_sort": bubble_sort,
    "insertion_sort": insertion_sort,
    "selection_sort": selection_sort,
}


def is_sorted(a: list) -> bool:  2 usages  new *
    """Проверка расположения элементов по неубыванию."""
    return all(a[i] <= a[i + 1] for i in range(len(a) - 1))
```

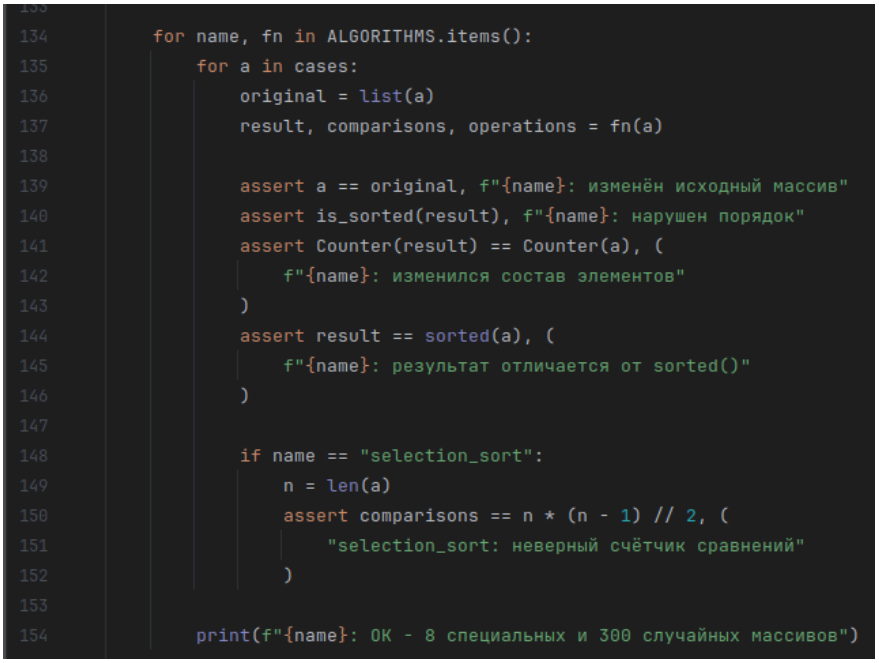
Рисунок 4 - Перечень сортировок и проверка упорядоченности массива



```
def self_check(seed: int) -> None:  new *
    """Граничные случаи, 300 случайных массивов и проверка счётчиков."""
    cases = [
        [],
        [7],
        [1, 2],
        [2, 1],
        [5] * 12,
        list(range(10)),
        list(range(9, -1, -1)),
        [-3, 0, -3, 2, -1],
    ]

    rng = random.Random(seed)
    for _ in range(300):
        cases.append([
            rng.randint(-100,  b: 100)
            for _ in range(rng.randint( a: 0,  b: 80))
        ])
```

Рисунок 5 - Подготовка специальных и случайных массивов для проверки



```
134         for name, fn in ALGORITHMS.items():
135             for a in cases:
136                 original = list(a)
137                 result, comparisons, operations = fn(a)
138
139                 assert a == original, f"{name}: изменён исходный массив"
140                 assert is_sorted(result), f"{name}: нарушен порядок"
141                 assert Counter(result) == Counter(a), (
142                     f"{name}: изменился состав элементов"
143                 )
144                 assert result == sorted(a), (
145                     f"{name}: результат отличается от sorted()"
146                 )
147
148                 if name == "selection_sort":
149                     n = len(a)
150                     assert comparisons == n * (n - 1) // 2, (
151                         "selection_sort: неверный счётчик сравнений"
152                     )
153
154             print(f"{name}: OK - 8 специальных и 300 случайных массивов")
```

Рисунок 6 - Проверка сохранности элементов, порядка сортировки и совпадения с sorted()

```
    assert bubble_sort([1, 2, 3])[1:] == (2, 0)
    assert bubble_sort([3, 2, 1])[1:] == (3, 3)
    assert insertion_sort([1, 2, 3])[1:] == (2, 0)
    assert insertion_sort([3, 2, 1])[1:] == (3, 5)

    print("Счётчики: OK")
    print("self_check: OK")


def main() -> None:  new *
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument( *name_or_flags: "--variant", type=int, required=True)
    args = parser.parse_args()

    seed = 30 + args.variant
    print(f"Вариант: {args.variant}; seed: {seed}")

    self_check(seed)

    sample = [3, 1, 2, 5, 4]
    print("\nИсходный массив:", sample)

    for name, fn in ALGORITHMS.items():
        result, comparisons, operations = fn(sample)
        label = "перемещения" if name == "insertion_sort" else "обмены"

        print(f"\n{name}:")
        print("Результат:", result)
        print(f"Сравнения: {comparisons}; {label}: {operations}")


if __name__ == "__main__":
    main()
```

Рисунок 7 - Проверка счётчиков операций, запуск программы и вывод результатов сортировок

```
Вариант: 16; seed: 46
bubble_sort: OK - 8 специальных и 300 случайных массивов
insertion_sort: OK - 8 специальных и 300 случайных массивов
selection_sort: OK - 8 специальных и 300 случайных массивов
Счётчики: OK
self_check: OK

Исходный массив: [3, 1, 2, 5, 4]

bubble_sort:
Результат: [1, 2, 3, 4, 5]
Сравнения: 7; обмены: 3

insertion_sort:
Результат: [1, 2, 3, 4, 5]
Сравнения: 6; перемещения: 6

selection_sort:
Результат: [1, 2, 3, 4, 5]
Сравнения: 10; обмены: 3
```

Рисунок 8 - Проверка правильности простых сортировок и счётчиков операций

Вывод: Все три сортировки прошли проверки на специальных случаях и 300 случайных массивах. Результаты совпали с sorted(), исходные массивы не изменились. Для сортировки выбором подтвердилось количество сравнений n(n - 1)/2.

3. Задание 2

Была реализована быстрая сортировка QuickSort. Для каждого участка массива случайно выбирался опорный элемент. Меньшие элементы размещались слева от него, остальные - справа. Границы участков, которые ещё требовалось отсортировать, сохранялись в стеке. Участки из одного элемента или без элементов пропускались. Сортировалась копия массива, поэтому исходные данные сохранялись.

```python
def quick_sort(a: list, rng=None, key=lambda x: x) -> list:  2 usages  new *
    """QuickSort: в среднем O(n log n), в худшем O(n²)."""
    if rng is None:
        rng = random

    b = list(a)
    stack = [(0, len(b))]

    while stack:
        lo, hi = stack.pop()

        if hi - lo <= 1:
            continue

        pivot_index = rng.randrange(lo, hi)
        b[pivot_index], b[hi - 1] = b[hi - 1], b[pivot_index]
        pivot = key(b[hi - 1])

        position = lo
        for j in range(lo, hi - 1):
            if key(b[j]) < pivot:
                b[position], b[j] = b[j], b[position]
                position += 1

        b[position], b[hi - 1] = b[hi - 1], b[position]

        if position - lo > 1:
            stack.append((lo, position))

        if hi - position - 1 > 1:
            stack.append((position + 1, hi))

    return b
```

Рисунок 9 - Реализация QuickSort со случайным выбором опорного элемента

```
self_check: OK
```

Рисунок 10 - Проверка работы QuickSort

```
Исходный: [3, 1, 2, 5, 4]
QuickSort: [1, 2, 3, 4, 5]
После вызова: [3, 1, 2, 5, 4]
```

Рисунок 11 - Результат быстрой сортировки

Вывод: Проверки прошли успешно. Массив был отсортирован, исходный список не изменился.

4. Задание 3

Были подготовлены упорядоченные, случайные и обратно упорядоченные массивы пяти размеров для каждой сортировки. Данные создавались заранее и использовались повторно между алгоритмами. Для каждого случая выполнялся один прогрев и пять измерений, после чего вычислялась медиана времени. Результаты и счётчики операций сохранялись в таблицу.

```
316  SIZES_QUADRATIC = [250, 500, 1_000, 2_000, 4_000]
317  SIZES_QUICK = [1_000, 2_000, 4_000, 8_000, 16_000]
318  REPEATS = 5
319
320
321  def bench(fn, data: list) -> tuple:  new *
322      """Один прогрев, затем пять повторов; результат - медиана времени."""
323      fn(list(data))
324      times = []
325
326      for _ in range(REPEATS):
327          arg = list(data)
328          start = time.perf_counter()
329          result = fn(arg)
330          times.append(time.perf_counter() - start)
331
332      return statistics.median(times), result
```

Рисунок 12 - Настройка размеров массивов и измерение времени сортировки

```
335  def run_benchmarks(seed: int) -> None:  new *
336      """Замеры на одинаковых данных и сохранение времени и операций."""
337      folder = Path(__file__).resolve().parent
338      sizes = sorted(set(SIZES_QUADRATIC + SIZES_QUICK))
339      datasets = {n: make_inputs(n, seed) for n in sizes}
340      pivot_rng = random.Random(seed)
341
342      plans = [
343          ("bubble_sort", bubble_sort, SIZES_QUADRATIC),
344          ("insertion_sort", insertion_sort, SIZES_QUADRATIC),
345          ("selection_sort", selection_sort, SIZES_QUADRATIC),
346          ("quick_sort", lambda a: quick_sort(a, pivot_rng), SIZES_QUICK),
347      ]
348
349      print(f"Вариант: {seed - 30}; seed: {seed}")
350      print(f"Python: {platform.python_version()}")
351      print(f"Методика: 1 прогрев, {REPEATS} повторов, медиана")
352      rows = []
```

Рисунок 13 - Подготовка данных и плана эксперимента

```
    for name, fn, plan_sizes in plans:
        print(f"\n{name}:")
        for n in plan_sizes:
            for cls, data in datasets[n].items():
                seconds, result = bench(fn, data)
                comparisons = operations = ""
                label = ""
                details = ""

                if name == "quick_sort":
                    sorted_data = result
                else:
                    sorted_data, comparisons, operations = result
                    label = (
                        "перемещения" if name == "insertion_sort" else "обмены"
                    )
                    details = (
                        f"; сравнения={comparisons}; {label}={operations}"
                    )

                assert sorted_data == datasets[n]["упорядоченный"]
                print(f"n={n:>6}; {cls:<13}; t={seconds:.9f} с{details}")
                rows.append({
                    "algorithm": name, "input": cls, "n": n,
                    "seconds": seconds, "comparisons": comparisons,
                    "operations": operations, "operation_type": label,
                })
```

Рисунок 14 - Сбор результатов замеров и счётчиков операций

```
        with (folder / "results.csv").open(
            mode: "w", newline="", encoding="utf-8-sig"
        ) as file:
            writer = csv.DictWriter(file, fieldnames=list(rows[0]))
            writer.writeheader()
            writer.writerows(rows)

        print(f"\nСохранено {len(rows)} строк в results.csv")
        print("Замеры завершены")
```

Рисунок 15 - Сохранение результатов эксперимента в таблицу

```
Architecture:                         x86_64
CPU op-mode(s):                       32-bit, 64-bit
Address sizes:                        39 bits physical, 48 bits virtual
Byte Order:                           Little Endian
CPU(s):                               24
On-line CPU(s) list:                  0-23
Vendor ID:                            GenuineIntel
Model name:                           Intel(R) Core(TM) i7-14650HX
CPU family:                           6
Model:                                183
Thread(s) per core:                   2
Core(s) per socket:                   16
Socket(s):                            1
Stepping:                             1
CPU(s) scaling MHz:                   28%
CPU max MHz:                          5200.0000
```

Рисунок 16 - Характеристики процессора, на котором выполнялись замеры

1	bubble_sort	упорядоченный	250	1.0849000318557955e-05	249	0	обмены
2	bubble_sort	случайный	250	0.0019361670001671882	31070	15651	обмены
3	bubble_sort	обратный	250	0.002322586000445881	31125	31125	обмены
4	bubble_sort	упорядоченный	500	2.2837000869913027e-05	499	0	обмены
5	bubble_sort	случайный	500	0.007898759000454447	124560	61842	обмены
6	bubble_sort	обратный	500	0.0098356149992469	124750	124750	обмены
7	bubble_sort	упорядоченный	1000	5.1120001444360246-05	999	0	обмены
8	bubble_sort	случайный	1000	0.03396961899852613	499247	241248	обмены
9	bubble_sort	обратный	1000	0.04161801400005061	499500	499500	обмены
10	bubble_sort	упорядоченный	2000	0.00010486300016054884	1999	0	обмены
11	bubble_sort	случайный	2000	0.14357519980113402	1997047	1001553	обмены
12	bubble_sort	обратный	2000	0.17031077399951755	1999000	1999000	обмены
13	bubble_sort	упорядоченный	4000	0.00021127800027898047	3999	0	обмены
14	bubble_sort	случайный	4000	0.5852824730009161	7986065	4029437	обмены
15	bubble_sort	обратный	4000	0.7005571070003498	7998000	7997997	обмены
16	insertion_sort	упорядоченный	250	1.9713999790837988e-05	249	0	перемещения
17	insertion_sort	случайный	250	0.0009438590004720027	15895	15898	перемещения
18	insertion_sort	обратный	250	0.0018973179994645761	31125	31374	перемещения
19	insertion_sort	упорядоченный	500	3.976200059696566e-05	499	0	перемещения
20	insertion_sort	случайный	500	0.003937002000384382	62338	62335	перемещения
21	insertion_sort	обратный	500	0.007697429999097949	124750	125249	перемещения
22	insertion_sort	упорядоченный	1000	8.367800000996795e-05	999	0	перемещения
23	insertion_sort	случайный	1000	0.015685554999436135	242243	242241	перемещения
24	insertion_sort	обратный	1000	0.031608312001480954	499500	500499	перемещения
25	insertion_sort	упорядоченный	2000	0.00016328199853887782	1999	0	перемещения
26	insertion_sort	случайный	2000	0.06579696500011778	1003542	1003542	перемещения
27	insertion_sort	обратный	2000	0.12971208399903844	1999000	2000999	перемещения
28	insertion_sort	упорядоченный	4000	0.0003354959990247153	3999	0	перемещения
29	insertion_sort	случайный	4000	0.26625141400108987	4033430	4033428	перемещения
30	insertion_sort	обратный	4000	0.5245476370000688	7998000	8001996	перемещения
31	selection_sort	упорядоченный	250	0.0013085000009596115	31125	0	обмены
32	selection_sort	случайный	250	0.001342645000477205	31125	243	обмены
33	selection_sort	обратный	250	0.001386683999953675	31125	125	обмены
34	selection_sort	упорядоченный	500	0.005721860001358436	124750	0	обмены

Рисунок 17 - Фрагмент таблицы времени выполнения сортировок и количества

Вывод: На упорядоченных массивах пузырёк и вставки работали значительно быстрее, чем на случайных и обратных. При увеличении размера неупорядоченных массивов время их обработки заметно возрастало.

5. Задание 4

Стабильность сортировок проверялась на парах из числа и буквенной метки. Сравнение выполнялось только по числу. После сортировки проверялось, сохранился ли исходный порядок меток у элементов с одинаковыми числами.

```python
def stability_demo(seed: int = 46) -> None:  1 usage  new *
    """Проверка порядка меток при сортировке только по ключу."""
    pairs = [
        (2, "a"), (1, "b"), (2, "c"),
        (1, "d"), (2, "e"), (1, "f"),
    ]
    expected = sorted(pairs, key=lambda p: p[0])
    rng = random.Random(seed)

    sorters = [
        ("bubble_sort", bubble_sort),
        ("insertion_sort", insertion_sort),
        ("selection_sort", selection_sort),
        ("quick_sort", quick_sort),
    ]

    print("Исходный массив:", pairs)

    for name, fn in sorters:
        if name == "quick_sort":
            result = fn(pairs, rng, key=lambda p: p[0])
        else:
            result = fn(pairs, key=lambda p: p[0])[0]

        assert is_sorted([p[0] for p in result])
        assert Counter(result) == Counter(pairs)

        stable = result == expected
        if name in ("bubble_sort", "insertion_sort"):
            assert stable, f"{name}: нарушена стабильность"

        print(f"\n{name}: {result}")
        print(
            "Порядок меток сохранён:",
            "да" if stable else "нет",
        )
```

Рисунок 18 - Код проверки стабильности сортировок

```
Исходный массив: [(2, 'a'), (1, 'b'), (2, 'c'), (1, 'd'), (2, 'e'), (1, 'f')]

bubble_sort: [(1, 'b'), (1, 'd'), (1, 'f'), (2, 'a'), (2, 'c'), (2, 'e')]
Порядок меток сохранён: да

insertion_sort: [(1, 'b'), (1, 'd'), (1, 'f'), (2, 'a'), (2, 'c'), (2, 'e')]
Порядок меток сохранён: да

selection_sort: [(1, 'b'), (1, 'd'), (1, 'f'), (2, 'a'), (2, 'e'), (2, 'c')]
Порядок меток сохранён: нет

quick_sort: [(1, 'f'), (1, 'b'), (1, 'd'), (2, 'a'), (2, 'c'), (2, 'e')]
Порядок меток сохранён: нет
```

Рисунок 19 - Результаты проверки стабильности сортировок

Вывод: Пузырёк и вставки сохранили порядок элементов с одинаковыми ключами. Сортировки выбором и QuickSort изменили этот порядок.

6. Задание 5

По результатам измерений были построены графики зависимости времени выполнения четырёх сортировок от размера массива. На каждом графике представлены

упорядоченные, случайные и обратно упорядоченные массивы. Для построения использовалась медиана пяти измерений. Обе оси имеют логарифмическую шкалу, позволяющую сравнить рост времени при увеличении объёма данных.

```python
import csv
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt


folder = Path(__file__).resolve().parent

with (folder / "results.csv").open(
    encoding="utf-8-sig", newline=""
) as file:
    rows = list(csv.DictReader(file))

algorithms = {
    "bubble_sort": "Сортировка пузырьком",
    "insertion_sort": "Сортировка вставками",
    "selection_sort": "Сортировка выбором",
    "quick_sort": "Быстрая сортировка QuickSort",
}

input_types = list(dict.fromkeys(row["input"] for row in rows))

for name, title in algorithms.items():
    fig, ax = plt.subplots(figsize=(8, 5))

    for input_type in input_types:
        points = sorted(
            (int(row["n"]), float(row["seconds"]))
            for row in rows
            if row["algorithm"] == name
            and row["input"] == input_type
        )

        sizes = [n for n, seconds in points]
        times = [seconds for n, seconds in points]

        ax.plot( *args: sizes, times, marker="o", label=input_type)
```

Рисунок 20 - Чтение результатов замеров и построение графиков

```
    ax.set_xscale( value: "log", base=2)
    ax.set_yscale("log")
    ax.set_xticks(sizes, labels=[str(n) for n in sizes])
    ax.set_title(title)
    ax.set_xlabel("Размер массива n")
    ax.set_ylabel("Медиана времени, с")
    ax.grid( visible: True, which="both", alpha=0.3)
    ax.legend()

    fig.tight_layout()
    path = folder / f"{name}_time.png"
    fig.savefig(path, dpi=200)
    plt.close(fig)

    print("Сохранён график:", path.name)

print("Построено 4 графика")
```

Рисунок 21 - Оформление и сохранение графиков

```
Сохранён график: bubble_sort_time.png
Сохранён график: insertion_sort_time.png
Сохранён график: selection_sort_time.png
Сохранён график: quick_sort_time.png
Построено 4 графика
```

Рисунок 22 - Создание и сохранение графиков времени сортировок

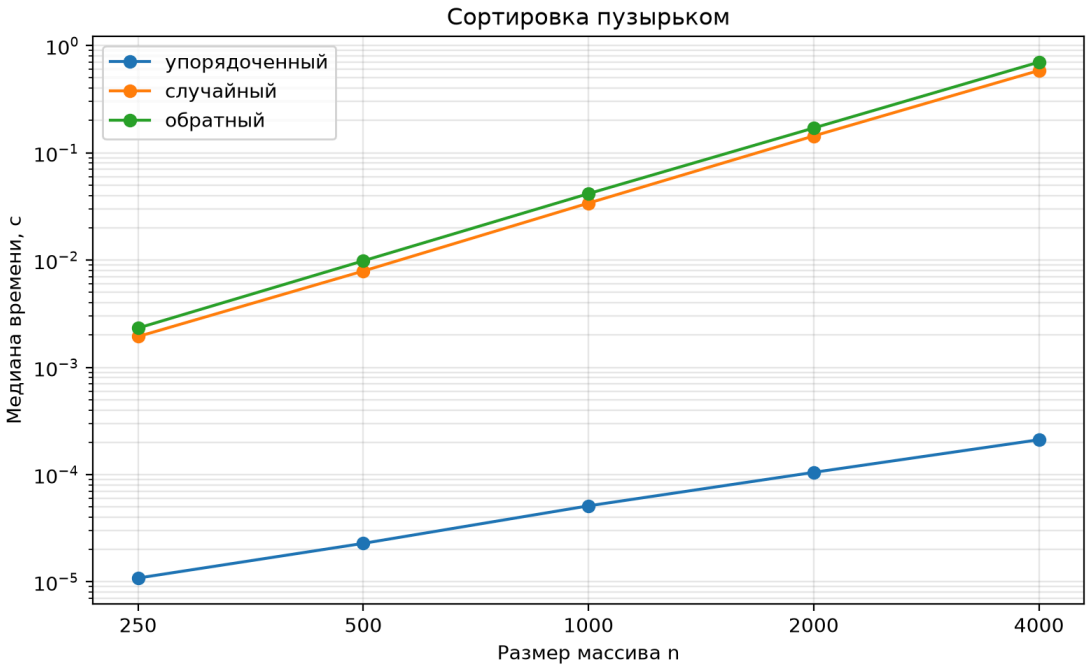


Рисунок 23 - Зависимость времени сортировки пузырьком от размера и типа массива

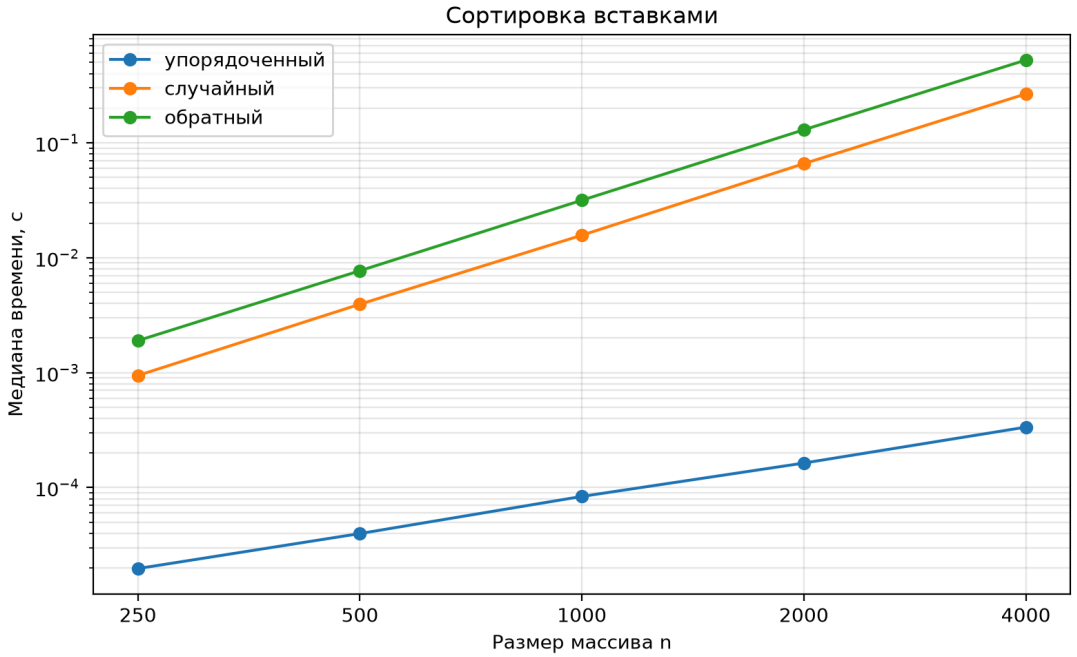


Рисунок 24 - Зависимость времени сортировки вставками от размера и типа массива

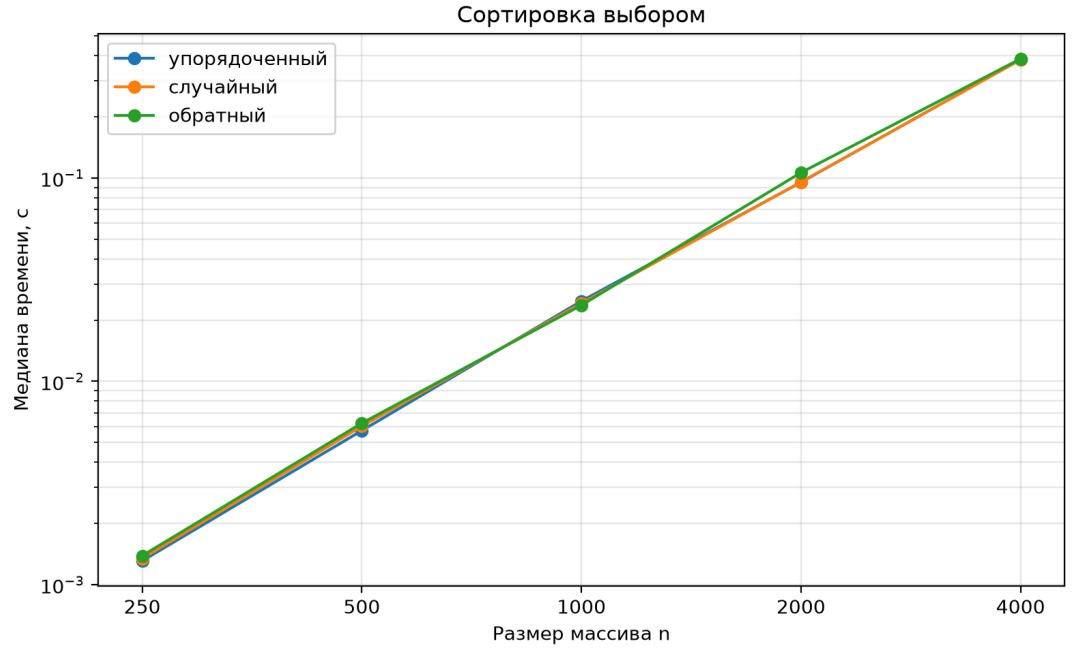


Рисунок 25 - Зависимость времени сортировки выбором от размера и типа массива

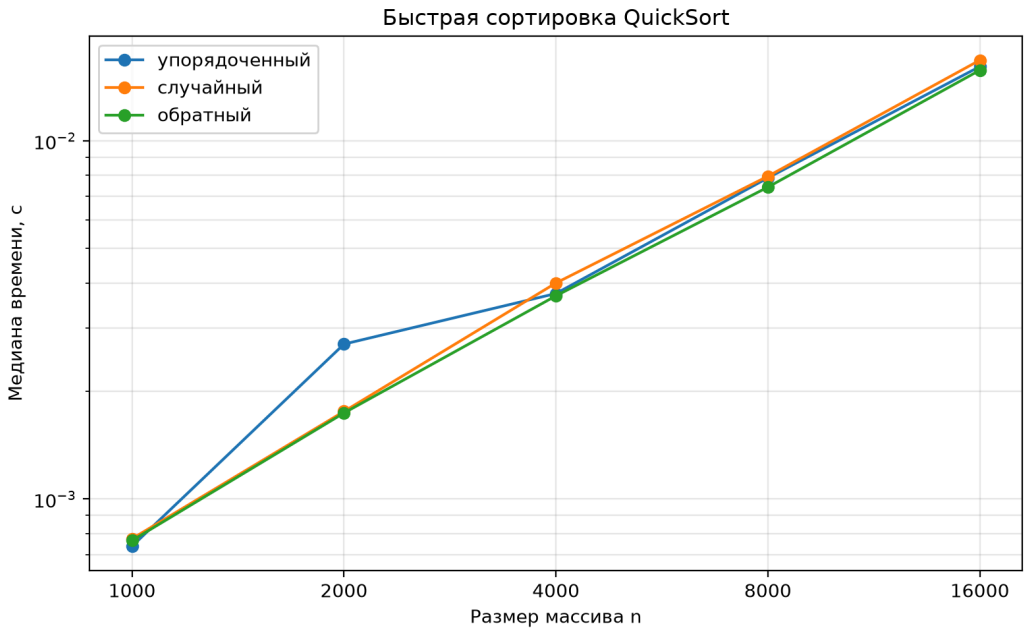
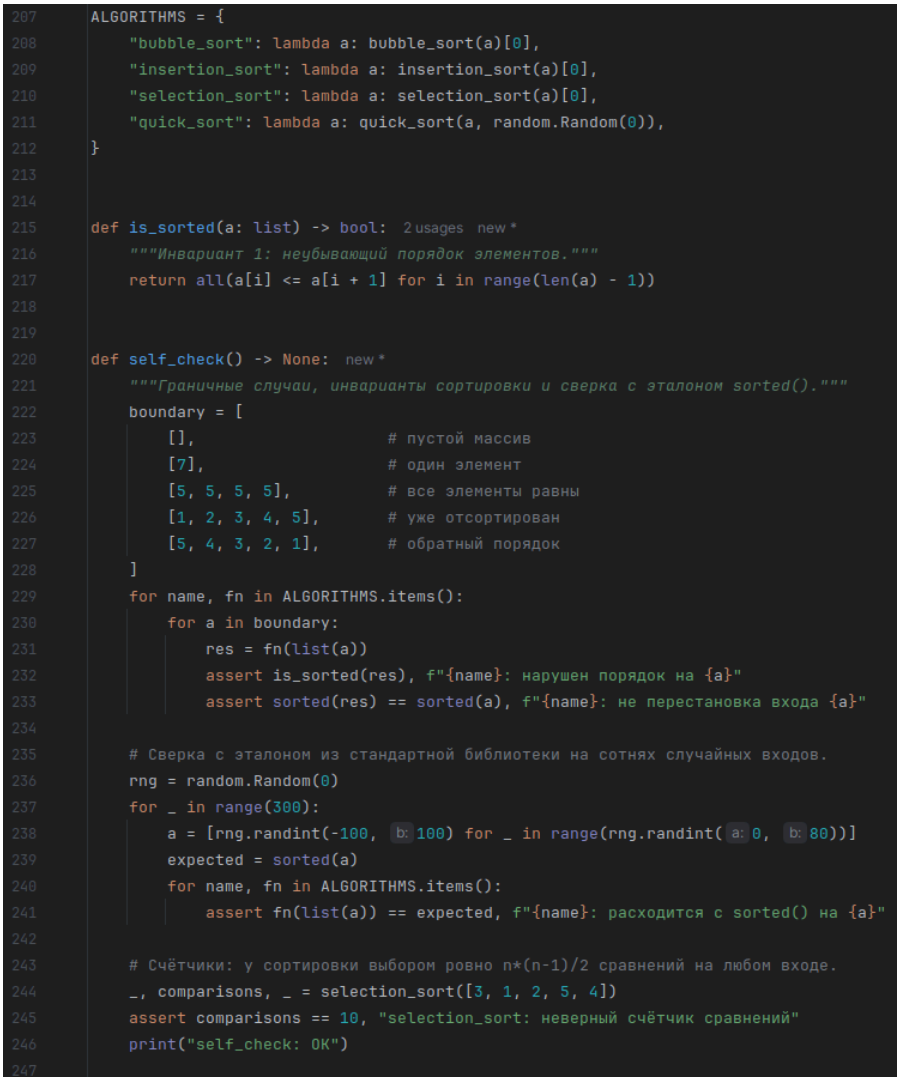


Рисунок 26 - Зависимость времени QuickSort от размера и типа массива

Вывод: Пузырёк и вставки быстрее обрабатывали упорядоченные массивы. Время сортировки выбором мало зависело от порядка элементов. На случайных и обратных массивах время QuickSort росло медленнее, чем у простых сортировок.

7. Задание 6

Из программы были удалены повторяющиеся блоки проверки и запуска. Затем все четыре сортировки были проверены на граничных случаях и 300 случайных массивах. Полученные результаты сравнивались с sorted(). Дополнительно проверялись стабильность сортировок и счётчик сравнений сортировки выбором.



```
ALGORITHMS = {
    "bubble_sort": lambda a: bubble_sort(a)[0],
    "insertion_sort": lambda a: insertion_sort(a)[0],
    "selection_sort": lambda a: selection_sort(a)[0],
    "quick_sort": lambda a: quick_sort(a, random.Random(0)),
}


def is_sorted(a: list) -> bool:  2 usages  new *
    """Инвариант 1: неубывающий порядок элементов."""
    return all(a[i] <= a[i + 1] for i in range(len(a) - 1))


def self_check() -> None:  new *
    """Граничные случаи, инварианты сортировки и сверка с эталоном sorted()."""
    boundary = [
        [],                  # пустой массив
        [7],                 # один элемент
        [5, 5, 5, 5],        # все элементы равны
        [1, 2, 3, 4, 5],     # уже отсортирован
        [5, 4, 3, 2, 1],     # обратный порядок
    ]
    for name, fn in ALGORITHMS.items():
        for a in boundary:
            res = fn(list(a))
            assert is_sorted(res), f"{name}: нарушен порядок на {a}"
            assert sorted(res) == sorted(a), f"{name}: не перестановка входа {a}"

    # Сверка с эталоном из стандартной библиотеки на сотнях случайных входов.
    rng = random.Random(0)
    for _ in range(300):
        a = [rng.randint(-100,  b: 100) for _ in range(rng.randint( a: 0,  b: 80))]
        expected = sorted(a)
        for name, fn in ALGORITHMS.items():
            assert fn(list(a)) == expected, f"{name}: расходится с sorted() на {a}"

    # Счётчики: у сортировки выбором ровно n*(n-1)/2 сравнений на любом входе.
    _, comparisons, _ = selection_sort([3, 1, 2, 5, 4])
    assert comparisons == 10, "selection_sort: неверный счётчик сравнений"
    print("self_check: OK")
```

Рисунок 27 - Код проверки правильности четырёх сортировок



```
def main() -> None:  new *
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument( *name_or_flags: "--variant", type=int, required=True, help="номер варианта")
    args = ap.parse_args()
    seed = 30 + args.variant
    random.seed(seed)
    self_check()
    stability_demo(seed)
    run_benchmarks(seed)


if __name__ == "__main__":
    main()
```

Рисунок 28 - Запуск проверок, демонстрации стабильности и замеров времени

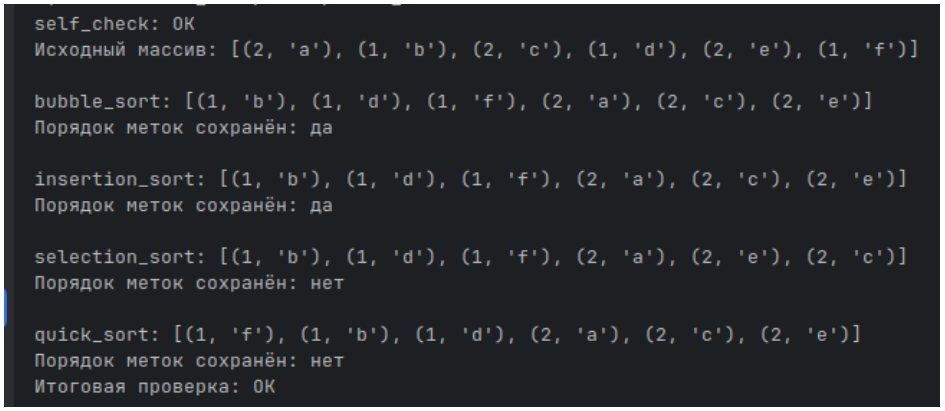


Рисунок 29 - Успешное выполнение итоговой проверки сортировок

Вывод: Все проверки прошли успешно. Результаты сортировок совпали с эталоном, ошибок не обнаружено.

Заключение

В ходе лабораторной работы были реализованы и проверены сортировки пузырьком, вставками, выбором и QuickSort. Были подсчитаны операции, измерено время работы на разных типах массивов и построены графики. Пузырёк и вставки быстрее обрабатывали упорядоченные данные, а время выбора мало зависело от порядка элементов. На неупорядоченных массивах QuickSort показал более медленный рост времени. Проверка стабильности показала, что пузырёк и вставки сохраняют порядок равных элементов, а выбор и QuickSort могут его изменять.